

Week 04 - Rest APIs

Lukáš Grolig et al.

Outline

- Softwarový tým a role
- Odpovědnosti v týmu
- T-shirt skillset
- Projekt a jeho fáze
- Řízení týmu: Kanban a Scrum
- JSON & JSON Schema
- REST API — Koncepty & Richardson Maturity Model
- REST API — Návrh, verzování & OpenAPI
- Implementace — Express, CORS, klientská strana
- Infrastruktura — Redis, testování, struktura projektu

Softwarový tým

Jak vypadá reálný softwarový tým?

Vývoj software není práce jednotlivce. Reálný tým se skládá z lidí s různými specializacemi, kteří společně pokrývají celý životní cyklus produktu — od analýzy požadavků, přes vývoj a testování, až po nasazení a provoz.

Role v týmu

- **Frontend developer** — vývoj uživatelského rozhraní (React)
- **Backend developer** — serverová logika, API, databáze
- **Fullstack developer** — kombinace frontendu i backendu
- **Manuální tester** — ruční ověřování funkčnosti aplikace
- **Automatizační tester** — psaní automatizovaných testů (Playwright)
- **DevOps engineer** — CI/CD, infrastruktura, monitoring, nasazení

Role v týmu (pokračování)

- **Scrum master** — facilitátor procesu, odstraňuje překážky týmu
- **Analytik** — sběr a specifikace požadavků, komunikace se zákazníkem
- **Product owner** — vlastník produktu, prioritizuje backlog, definuje vizi
- **Projektový manažer** — řízení projektu, rozpočet, harmonogram, rizika

Co je to T-shirt skillset?

Způsob, jak vizualizovat úroveň znalostí člena týmu v různých oblastech.

Oblast	Úroveň
Frontend	XL
Backend	M

- **XL** — expert, dokáže vést ostatní
- **L** — pokročilý, pracuje samostatně
- **M** — středně pokročilý, potřebuje občasnou pomoc
- **S** — začátečník, zvládne základní úkoly
- **XS** — povědomí, potřebuje vedení

T-shirt skillset — proč na tom záleží?

- Pomáhá identifikovat **silné stránky** a **mezery** v týmu
- Usnadňuje **plánování rozvoje** jednotlivců
- Podporuje budování **T-shaped** profilů — hluboká znalost v jedné oblasti + široký přehled
- Umožňuje lepší **rozložení práce** v týmu

Odpovědnosti v týmu

Každá role má jasně definované odpovědnosti:

Role	Hlavní odpovědnost
Frontend developer	UI komponenty, UX, responzivita
Backend developer	API, business logika, databáze
Tester	Kvalita, regresní testy, bug reporty
DevOps	Nasazení, monitoring, infrastruktura
Analytik	Požadavky, specifikace, akceptační kritéria
Product owner	Vize produktu, prioritizace backlogu
Scrum master	Proces, facilitace, odstranění překážek

Projekt a jeho fáze

Životní cyklus projektu

1. **Iniciace** — definice cílů, rozsahu a stakeholderů
2. **Plánování** — harmonogram, rozpočet, technologický stack
3. **Realizace** — vývoj, testování, iterativní dodávání
4. **Nasazení** — deployment, migrace dat, školení uživatelů
5. **Provoz a údržba** — monitoring, opravy chyb, rozvoj

Vnitřní řízení týmu

Kanban

- **Kontinuální proces** — práce plyne průběžně, bez pevných iterací
- **Prioritizovaný backlog** — úkoly seřazeny podle důležitosti
- Typy položek: **story** (nová funkcionality) a **defekt** (chyba)
- Posun pouze **zleva doprava** — vizualizace na Kanban boardu

Kanban board

Backlog	To Do	In Progress	Review	Done
Story #5 Bug #4	Story #3	Story #1 Bug #2	Story #2	Story #0 Bug #1

→
Posun pouze zleva doprava

- **WIP limit** — omezení počtu úkolů rozpracovaných současně
- Cíl: plynulý tok práce, minimalizace přepínání kontextu

Scrum

- Iterace probíhají v **sprintech** (typicky 1–4 týdny)
- **Backlog** — prioritizovaný seznam požadavků
- Na začátku sprintu tým vybere položky do **sprint backlogu**
- Na konci sprintu vzniká **potenciálně nasaditelný inkrement**

Scrum ceremonie

- **Standup (Daily Scrum)**
 - Každodenní krátká schůzka (max 15 min)
 - Co jsem udělal? Co budu dělat? Jaké mám překážky?
- **Refinement (Grooming)**
 - Upřesnění a odhad položek v backlogu
 - Příprava stories pro další sprint

Scrum ceremonie (pokračování)

- **Retrospektiva**
 - Reflexe uplynulého sprintu
 - Co fungovalo? Co zlepšit? Jaké akce podnikneme?
- **Sprint Review (Demo)**
 - Prezentace hotové práce stakeholderům
 - Získání zpětné vazby

Kanban vs Scrum — srovnání

	Kanban	Scrum
Iterace	Kontinuální tok	Sprinty (1–4 týdny)
Role	Nejsou předepsané	Scrum Master, PO, Tým
Plánování	Průběžné	Na začátku sprintu
Změny	Kdykoliv	Až v dalším sprintu
Metriky	Lead time, throughput	Velocity, burndown
Vhodné pro	Údržbu, support	Nový vývoj, produkty

API

Součásti aplikace

- Frontend / UI
- Backend / služby
- Databáze

Struktura aplikace

- Excalidraw

JSON

JSON (JavaScript Object Notation)

- Není to značkovací jazyk
- Používá se hlavně pro výměnu dat
- Lze komprimovat, protože bílé znaky jsou ignorovány
- Mime type je `application/json`
- Kompaktnější než XML
- Založen na podmnožině JavaScriptu (objektová notace)

Struktura JSON

- Dvě základní struktury:
 - i. Páry klíč-hodnota `{"name": "John"}`
 - Hodnoty mohou být: boolean, int, string, objekt, pole a `null`
 - Obvykle implementováno jako: *objekt, slovník, hash-tabulka*
 - ii. Uspořádané seznamy hodnot `{"people": [ {"name": "John"}, {"name": "Jane"} ]}`
 - Obvykle implementováno jako: *pole, seznam, vektor, sekvence*
- Komentáře **nejsou** povoleny


```
{ // object
  "squadName": "Super hero squad", // string
  "formed": 2016, // int
  "active": true, // boolean
  "disbanded": null, // null
  "powerLevel": 9.6, // float
  "nicknames": ["incredibles", "saviors", "a-team"], // array
  "members": [ // array
    {
      "name": "Molecule Man",
      "age": 29,
      "powers": ["Radiation resistance", "Radiation blast"]
    },
    {
      "name": "Madame Uppercut",
      "age": 39,
      "powers": ["Million tonne punch"]
    }
  ]
}
```

JSON Schema

Často je potřeba validovat JSON objekty — ověřit, že požadované vlastnosti jsou přítomné a že jsou splněna další omezení (např. cena nesmí být nižší než jeden dolar). Validace se typicky provádí pomocí JSON Schema.

- JSON Schema je vyjádřeno schématem, které je samo o sobě JSON objekt
- JSON Schema je spravováno na <http://json-schema.org>
- Popisuje existující datový formát
- Nabízí jasnou, lidsky i strojově čitelnou dokumentaci
- Poskytuje kompletní strukturální validaci, užitečnou pro automatizované testování a validaci dat od klientů

- Doporučuje se použít [generátor](#)
 - Výsledky je potřeba dále zkontrolovat

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "squadName": {
      "type": "string"
    },
    "powerLevel": {
      "type": "number"
    },
    "nicknames": {
      "type": "array",
      "items": [
        {
          "type": "string"
        }
      ]
    }
  }
}
...
```

REST API — Koncepty

Co je Web API?

Služba, která poskytuje rozhraní (např. REST API) dalším službám pro změnu stavu služby nebo získání informací.

- API = Application Programming Interface (rozhraní pro programování aplikací)
- Tři hlavní typy API: Private, Partner a Public
- Implementováno různými způsoby:
 - RPC
 - SOAP
 - **REST**
 - gRPC
 - **GraphQL**

RESTful API

HTTP metody, URI a odpovědi v JSON tvoří REST API.

REST se používá k provádění **CRUD** operací nad daty.

Pomocí URI specifikujeme zdroj: `/posts/1`

Pomocí *HTTP sloves* určujeme, co se má s daty stát `VERB /posts/1`

- **POST** - **Create** - neidempotentní - opakovaná volání vytvoří nové zdroje
- **GET** - **Retrieve** - bezpečné, idempotentní - opakovaná volání se stejným vstupem vrátí stejný výsledek a nemodifikují DB
- **PUT** - **Update** (celý) - idempotentní - nahrazuje celý zdroj
- **PATCH** - **Update** (částečný) - nemusí být idempotentní - modifikuje pouze zadaná pole
- **DELETE** - **Delete** - idempotentní

Dodatečné zpracování je specifikováno query parametry a zpracováno serverem.

To zahrnuje řazení, filtrování nebo omezení počtu výsledků.

Příklad

- `https://jsonplaceholder.typicode.com/posts/{id}/comments?sort=asc&limit=10`
- Rozeberme si tento příklad
 - i. `https://jsonplaceholder.typicode.com` -- HOST
 - ii. `/posts` -- Podstatné jméno specifikující zdroj. **Vždy** používejte podstatná jména, nikdy slovesa jako `getAllPosts`.
 - iii. `{id}` -- Identifikátor. V reálném požadavku se nahradí identifikátorem, např. `1`. Pokud je vynechán, požadavek by měl vrátit všechny entity daného zdroje.
 - iv. `/comments` -- Vnořená entita. V tomto případě požadujeme komentáře příspěvku s `{id}`.
 - v. `?sort=asc` -- Požadavek na vzestupné řazení výsledků.
 - vi. `&limit=10` -- Další parametry jsou odděleny znakem `&`. `limit=10` znamená vrátit maximálně 10 výsledků.

Richardson Maturity Model

Úroveň 0 - The Swamp of POX

- Nevyužívá žádné z možností URI, HTTP metod ani HATEOAS.
- Jeden endpoint pro vše, akce se rozlišují obsahem požadavku

Jak to zlepšit

- Používejte spinal-case pro URL (RFC 3986) = ŽÁDNÉ PODTRŽÍTKA
- Používejte malá písmena
- NEPOUŽÍVEJTE umělé přípony souborů (např. .json)

Úroveň 0 — Příklad

 Špatně – jeden endpoint, akce v těle požadavku

POST /api

```
{ "action": "getUser", "userId": 123 }
```

POST /api

```
{ "action": "createUser", "name": "John" }
```

POST /api

```
{ "action": "deleteUser", "userId": 123 }
```

 Správně – alespoň oddělené URL

POST /getUser

POST /createUser

POST /deleteUser

Úroveň 1 - Zdroje (Resources)

- Použití různých URL pro interakci s různými zdroji aplikace
- Použití sloves v URI, což je špatná praxe, ale na úrovni 1 stále přítomné

Doporučená vylepšení

- URI by nemělo končit lomítkem (/)
- Lomítko (/) se musí používat pro hierarchické vztahy
- Konzistentní použití jednotného a množného čísla

Úroveň 1 — Příklad

 Špatně – slovesa v URL, nekonzistentní pojmenování

POST /getUser/123

POST /create_user

POST /Delete_Users/123

GET /user/123/get-posts/

 Správně – zdroje jako podstatná jména, konzistentní formát

POST /users/123

POST /users

POST /users/123

GET /users/123/posts

Stále ale používá pouze POST pro vše — to řeší úroveň 2.

Úroveň 2 - Metody

- Používejte HTTP slovesa implicitně — žádná explicitní slovesa v URL
- Pro ne-RESTové operace použijte metodu POST

Úroveň 2 — Příklad

❌ Špatně – slovesa v URL, špatné HTTP metody

```
GET    /getUser/123
POST   /createUser
POST   /deleteUser/123
POST   /updateUser/123
```

✅ Správně – HTTP metody určují akci

```
GET    /users/123      # načtení uživatele
POST    /users        # vytvoření uživatele
DELETE  /users/123    # smazání uživatele
PUT     /users/123    # aktualizace uživatele
PATCH  /users/123    # částečná aktualizace
```

Úroveň 3 - Hypermedia Controls

- Úroveň 3 využívá HATEOAS (Hypermedia As Transfer Engine Of Application State)
- Usnadňuje navigaci mezi zdrojem a jeho dostupnými akcemi
- Vývojář nemusí vědět, jak interagovat s aplikací pro různé akce, protože veškerá metadata budou součástí odpovědí ze serveru

Úroveň 3 — Příklad

```
# ❌ Špatně – klient musí znát všechny URL předem
GET /users/123
# Odpověď: { "name": "John Doe", "age": 30 }
# Klient musí hardcodovat: /users/123/posts, /users/123/address
```

```
// ✅ Správně – odpověď obsahuje navigační odkazy
{
  "name": "John Doe",
  "links": [
    { "rel": "self", "href": "/users/123" },
    { "rel": "posts", "href": "/users/123/posts" },
    { "rel": "address", "href": "/users/123/address" },
    { "rel": "delete", "href": "/users/123", "method": "DELETE" }
  ]
}
```


Stavové kódy — Úspěch

- **200 OK** — Obecný úspěch. Tělo odpovědi závisí na metodě (GET vrací entitu, POST vrací výsledek akce).
- **201 Created** — Zdroj vytvořen (typicky POST). Odpověď obsahuje URI nového zdroje. Pokud je vytvoření zpožděno, použijte 202.
- **202 Accepted** — Požadavek přijat ke zpracování, ale zatím nedokončen. Používá se pro dlouho trvající akce.
- **204 No Content** — Úspěch bez těla odpovědi. Typicky pro PUT, POST nebo DELETE.

Stavové kódy — Chyby klienta

- **400 Bad Request** — Obecná chyba klienta: chybná syntaxe, neplatné parametry. Klient by NEMĚL opakovat požadavek bez úprav.
- **401 Unauthorized** — Chybějící nebo neplatná autentizace. Klient může zkusit znovu se správnými přihlašovacími údaji.
- **403 Forbidden** — Požadavek je validní, ale uživatel nemá oprávnění k danému zdroji.
- **404 Not Found** — Zdroj na dané URI neexistuje. V budoucnu může být dostupný.

Kompletní seznam na [dokumentaci](#).

Stavové kódy — Chyby serveru

- **500 Internal Server Error** — Obecná chyba na straně serveru. Nikdy to není chyba klienta.
 - NEZOBRAZUJTE klientovi interní detaily
 - Zalogujte výjimku a vraťte korelační ID
 - Klient může rozumně zkusit stejný požadavek znovu

REST API — Návrh

Návrh REST API

1. Podstatná jména, ne slovesa
2. Bezstavovost (statelessness)

Viz: <https://github.com/Microsoft/api-guidelines/blob/master/Guidelines.md>

Pojmenování

Zde je několik příkladů správného a špatného pojmenování.

DO

GET /storage/1/files

GET /phone-devices/1

POST /subscription

GET /files/

DONT

GET /get_files/1

GET /phone_devices or GET /phoneDevices

POST /subscribe

GET /list_files

Query parametry

Měli byste implementovat také další parametry pro běžné akce:

- **Paging** — `GET /posts?page=2&limit=20`
- **Filtering** — `GET /posts?status=published&author=john`
- **Sorting** — `GET /posts?sort=created_at&order=desc`
- **Searching** — `GET /posts?q=javascript`

Verzování

- Verzování lze chápat jako jinou reprezentaci zdroje
- Existují 2 běžně používané přístupy — verzování v hlavičce a v URL

Verzování v hlavičce

- Nezapomeňte nastavit výchozí verzi

Vlastní hlavička

Přidání vlastní hlavičky X-API-VERSION (nebo jiné dle volby) klientem umožňuje službě směřovat požadavek na správný endpoint

```
X-API-VERSION: v4
```

Accept hlavička

Použití Accept hlavičky pro specifikaci verze, např.

```
Accept: application/vnd.hashmapinc.v2+json
```

Verzování v URL

Vložení verze přímo do URL, např.

```
POST /v2/user
```

REST odpovědi

Existuje mnoho standardů pro strukturu odpovědí REST API.

- [JSON:API](#)
- [JSend](#)
- [HAL](#)
- [rfc7807](#)

GET /posts/2

```
{  
  "status" : "success",  
  "data" : { "post" : { "id" : 2, "title" : "Another blog post", "body" : "More content" } }  
}
```

OpenAPI/Swagger

Proč dokumentovat API?

- Bez dokumentace musí konzumenti hádat, jak vaše API funguje
- Dokumentace slouží jako **kontrakt** mezi frontend a backend týmy
- Umožňuje **generování kódu** — klientská SDK, serverové stuby, typy
- Umožňuje **automatizované testování** — validace požadavků/odpovědí proti specifikaci
- Živá dokumentace, která zůstává v souladu s kódem

Dokumentace API

OpenAPI Specification (OAS) definuje standardní, jazykově nezávislé rozhraní pro RESTful API, které umožňuje lidem i strojům objevit a pochopit možnosti služby bez přístupu ke zdrojovému kódu, dokumentaci nebo analýzy síťového provozu.

Lze psát jako YAML nebo JSON.

Lze použít ke generování dokumentace API ve formě aplikace Swagger.

Info objekt

Objekt poskytuje metadata o API. Metadata MOHOU být použita klienty podle potřeby a MOHOU být zobrazena v editorech nebo nástrojích pro generování dokumentace.

```
title: Sample Pet Store App
description: This is a sample server for a pet store.
termsOfService: http://example.com/terms/
contact:
  name: API Support
  url: http://www.example.com/support
  email: support@example.com
license:
  name: Apache 2.0
  url: https://www.apache.org/licenses/LICENSE-2.0.html
version: 1.0.1
```

Component objekt — Schema

Obsahuje sadu znovupoužitelných objektů pro různé aspekty OAS. Všechny objekty definované uvnitř components objektu nemají žádný vliv na API, pokud na ně není explicitně odkazováno z vlastností mimo components objekt.

```
components:
  schemas:
    GeneralError:
      type: object
      properties:
        code:
          type: integer
          format: int32
        message:
          type: string
    Category:
      type: object
      properties:
        id:
          type: integer
          format: int64
        name:
          type: string
    Tag:
      type: object
      properties:
        id:
          type: integer
          format: int64
        name:
          type: string
```


Server objekt

servers:

- url: <https://development.gigantic-server.com/v1>
description: Development server
- url: <https://staging.gigantic-server.com/v1>
description: Staging server
- url: <https://api.gigantic-server.com/v1>
description: Production server

Path objekt

```
/pets:  
  get:  
    description: Returns all pets from the system that the user has access to  
    responses:  
      '200':  
        description: A list of pets.  
        content:  
          application/json:  
            schema:  
              type: array  
              items:  
                $ref: '#/components/schemas/Pet'
```

Path item objekt

Popisuje operace dostupné na jedné cestě.

```
get:
  description: Returns pets based on ID
  summary: Find pets by ID
  operationId: getPetsById
  responses:
    '200':
      description: pet response
      content:
        '*/*' :
          schema:
            type: array
            items:
              $ref: '#/components/schemas/Pet'
  default:
    description: error payload
    content:
      'text/html':
        schema:
          $ref: '#/components/schemas/ErrorMessage'
parameters:
- name: id
  in: path
  description: ID of pet to use
  required: true
  schema:
    type: array
    items:
      type: string
  style: simple
```

OpenAPI v praxi

Dva přístupy k vytvoření OpenAPI specifikace:

1. Code-first — napíšete API, specifikaci vygenerujete z kódu

- `swagger-jsdoc` + `swagger-ui-express` pro Express
- Anotace/dekorátory v route handlerech

2. Design-first — napíšete specifikaci, kód vygenerujete z ní

- `openapi-generator` — generuje klientská SDK a serverové stuby
- Užitečné, když frontend a backend týmy pracují paralelně

Nástroje: [Swagger Editor](#), Swagger UI, Redoc

Implementace

Express.js 5

- Express 5 je nejnovější major verze (2024)
- Route handlers nyní podporují `async/await` — vyhozené chyby a zamítnuté promisy se zpracují automaticky
- Není potřeba `express-async-errors` ani ruční `try/catch` wrappery
- `req.params`, `req.query`, `req.body` vrací `undefined` místo vyhození chyby při chybějících hodnotách
- `app.listen()` vrací Promise

```
npm install express@5
```

REST API pomocí Express 5

```
import express from 'express';

const app = express();

app.use(express.json());

app.get('/user/:id', async (req, res) => {
  res.send(`user ${req.params.id}`);
});

app.post('/profile', async (req, res) => {
  console.log(req.body);
  res.json(req.body);
});

await app.listen(3000);
console.log('Server running on port 3000');
```

CORS

Ještě jedna věc před psaním vlastních REST API

CORS byl vytvořen pro umožnění servírování REST API různým originům při zachování kontroly a bezpečnosti.

Server může v hlavičkách odpovědi specifikovat, které originy smí posílat požadavky.

Pokud prohlížeč zjistí, že server neodpověděl se správnými hlavičkami, požadavek zruší.

origin — část URL, ze které požadavek pochází, např. `app.example.com` (schéma, hostname, port)

*CORS se používá jako náhrada za **JSONP** (JSON obalený callbackem)*

CORS — jak to funguje

1. Origin se představí metodou `OPTIONS`, tzv. **pre-flight** požadavek.
 - Představení zahrnuje, co klientský kód hodlá požadovat (metoda a URI)
 - Prohlížeč to dělá automaticky
 - Klient si této kontroly ani není vědom (dokud nepřestane fungovat)
2. Server odpoví na `OPTIONS` požadavek — informuje prohlížeč, zda takový požadavek povolí
3. Prohlížeč zkontroluje odpověď, zda je metoda a origin povolena
4. Pokud kontroly projdou, provede se skutečný požadavek
 - Kontroly se opakují i na skutečném požadavku
 - Dobře nastavený server odpoví kladně pouze pokud povolí následný požadavek
 - Mnoho serverů povolí jakýkoliv `OPTIONS` požadavek a pak odmítne ten skutečný

CORS — Implementace

CORS má mnoho nuancí, které zde nejsou uvedeny.

Následující kód nakonfiguruje Express pro použití správných hlaviček a vše by mělo fungovat hladce.

```
import express from 'express';  
import cors from 'cors';  
  
const app = express();  
app.use(cors());
```

Asynchronní REST API

- Neposkytuje data okamžitě.
- Vrací 201 Created s Location hlavičkou nového zdroje
- Klient provádí polling na novém zdroji
- Měl by obsahovat stav průběhu
- Alternativně může existovat dočasný zdroj vracející 303 (See other) po dokončení

```
GET /api/v1/report/523
{
  "reportDate": "2021-01-01",
  "reportType": "COFFEE_SALES",
  "completed": true,
  "pdfUri": "https://s3.eu-central-1.amazonaws.com/.../report-123.pdf"
}
```

Klientská strana

Komunikace s REST API

- Všimněte si hlavičky požadavku, která specifikuje, jak má server interpretovat tělo
- `fetch` je vestavěný v prohlížeči i Node.js (od v18) — není potřeba externí knihovna

```
fetch(  
  'https://jsonplaceholder.typicode.com/posts',  
  {  
    method: 'POST',  
    body: JSON.stringify({  
      title: 'foo',  
      body: 'bar',  
      userId: 1,  
    }),  
    headers: {  
      'Content-type': 'application/json; charset=UTF-8',  
    },  
  })  
  .then((response) => response.json())  
  .then((json) => console.log(json));
```

Komunikace s REST API

- Pro vyhnutí se "callback hell"

```
try {
  const request = await fetch(
    'https://jsonplaceholder.typicode.com/posts',
    {
      method: 'POST',
      body: JSON.stringify({
        title: 'foo',
        body: 'bar',
        userId: 1,
      }),
      headers: {
        'Content-type': 'application/json; charset=UTF-8',
      },
    });
  const data = await request.json();
} catch (e) {}
```

Prostě použijte Tanstack Query, jak prezentoval Filip.

Infrastruktura

Redis

Co je Redis

- In-memory datové úložiště používané jako cache, vektorová databáze, dokumentová databáze, streaming engine a message broker.
- Vy ho budete používat hlavně jako cache

Instalace Redis

- Pro vývojové účely budeme používat Redis Server a Redis Insights

```
docker run -d --name redis-stack-server -p 6379:6379 redis/redis-stack-server:latest
```

Instalace Redis npm balíčků

```
npm install redis --save  
npm install @types/redis --save-dev
```

Redis — příklad

```
import { createClient } from 'redis';

const client = createClient(); // connect to localhost on port 6379

client.on('error', err => console.log('Redis Client Error', err));

await client.connect();

await client.set('key', 'value');
const value = await client.get('key');
```

Práce s mapou

```
await client.hSet('user-session:123', {  
  name: 'John',  
  surname: 'Smith',  
  company: 'Redis',  
  age: 29  
})  
  
let userSession = await client.hGetAll('user-session:123');  
console.log(JSON.stringify(userSession, null, 2));
```

Balíček IORedis

Alternativa k oficiálnímu balíčku `redis`. IORedis je vyspělejší, má vestavěnou podporu TypeScriptu a poskytuje bohatší API (auto-pipelining, Lua skripty, podpora clusterů).

```
npm install ioredis
```

Jak používat Redis jako cache

```
import express from 'express';
import Redis from 'ioredis';

const app = express();
const port = process.env.PORT || 3000;

// Create a Redis client
const redis = new Redis();

// Example of caching data
app.get('/cache', async (req, res) => {
  const cachedData = await redis.get('cachedData');

  if (cachedData) {
    // If data exists in the cache, return it
    res.send(JSON.parse(cachedData));
  } else {
    // If data is not in the cache, fetch it from the source
    const dataToCache = { message: 'Data to be cached' };
    await redis.set('cachedData', JSON.stringify(dataToCache), 'EX', 3600); // Cache for 1 hour
    res.send(dataToCache);
  }
});
```


Testování

Vývojářské testování

Pro vývoj API, kde je užitečné rychlé testování a prototypování, se hodí následující klienti:

- VS Code Rest Client nebo vestavěný klient ve Webstormu
- [Bruno](#) — open-source, offline-first API klient
- `curl` nebo `httpie` z příkazové řádky

Výkonnostní testy

- Podívejte se na k6

```
import http from 'k6/http';

export default function () {
  const url = 'http://test.k6.io/login';
  const payload = JSON.stringify({
    email: 'aaa',
    password: 'bbb',
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
    },
  };

  http.post(url, payload, params);
}
```

Struktura projektu

Více repozitářů

- Projekt na repozitář
- Oddělení projektů
- Oddělení deployment pipeline
- Těžší vývoj, údržba a onboarding lidí

Monorepo

- Více projektů v jednom repozitáři
- Snazší začátek
- Snazší spuštění vývojového prostředí

NX pro monorepa

```
npx create-nx-workspace@latest  
npx nx add @nx/express  
npx nx g @nx/express:app my-app-api --directory=apps/my-app-api  
npx nx serve my-app-api
```

To je vše.

Zdroje

- <https://en.wikipedia.org/wiki/URL>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
- <https://www.w3.org/2001/sw/wiki/REST>
- velmi doporučené čtení, pokud to myslíte s REST vážně